Lecture 22
Learning
Complex Distributions

Armando Solar-Lezama

n-gram models

The big brown bear scares the children with its roar

$P(scares \mid bear, brown)$

Probability of a word depends on the previous n words

Represented with a table: $P(w_i \mid w_{i-1}, w_{i-2}, \ldots, w_{i-n})$

Bigger n makes more accurate, but also more difficult to learn, requires much bigger table

Downsides

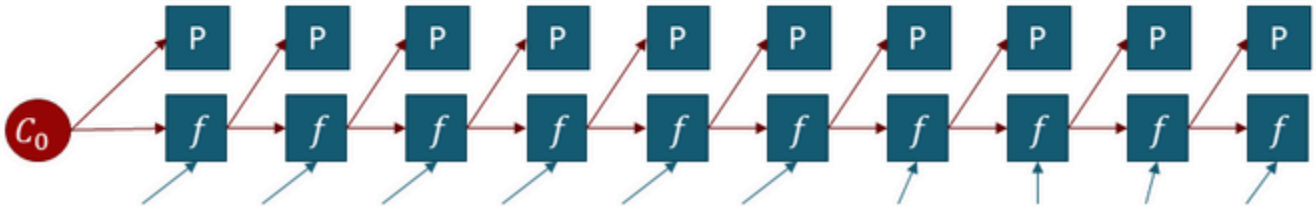
- some words require more context than others
- some words carry very little information

Recurrent models

Key idea: let the system figure out how to construct its own context

Now need to learn two interrelated functions

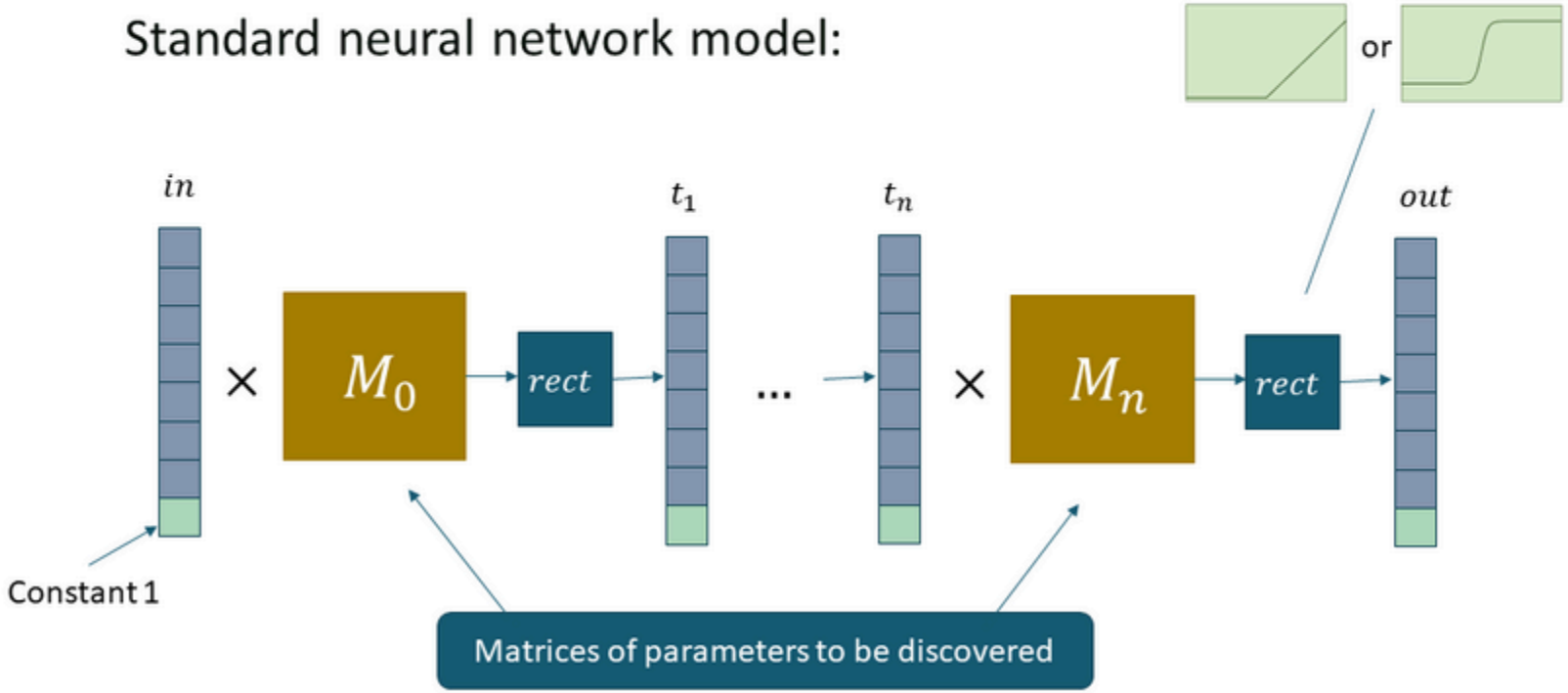
- $P(word_i \,|context_{i-1})$
- $context_i = f(word_i, context_{i-1})$



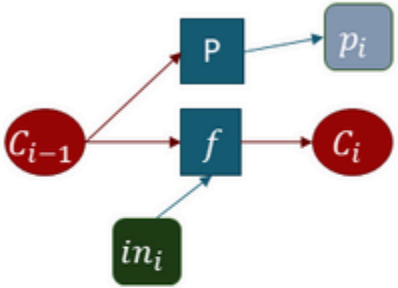
Neural Networks

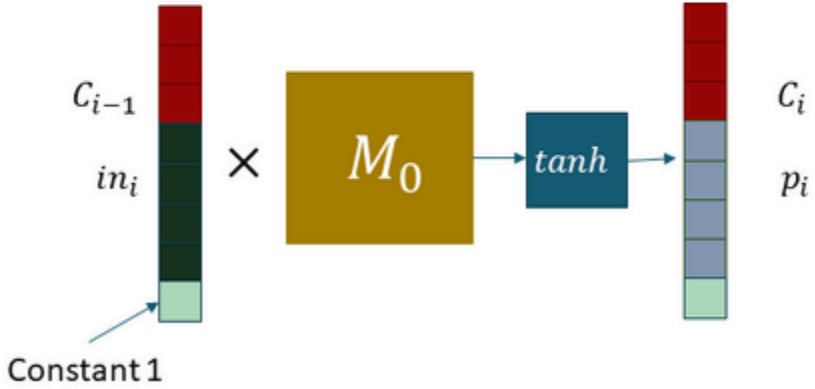
Most popular way of representing recurrent models

Standard neural network model:

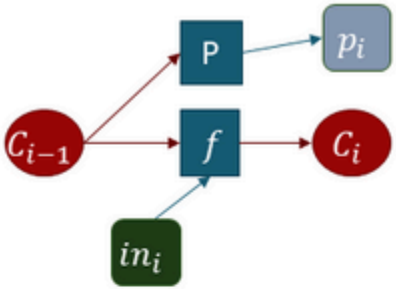


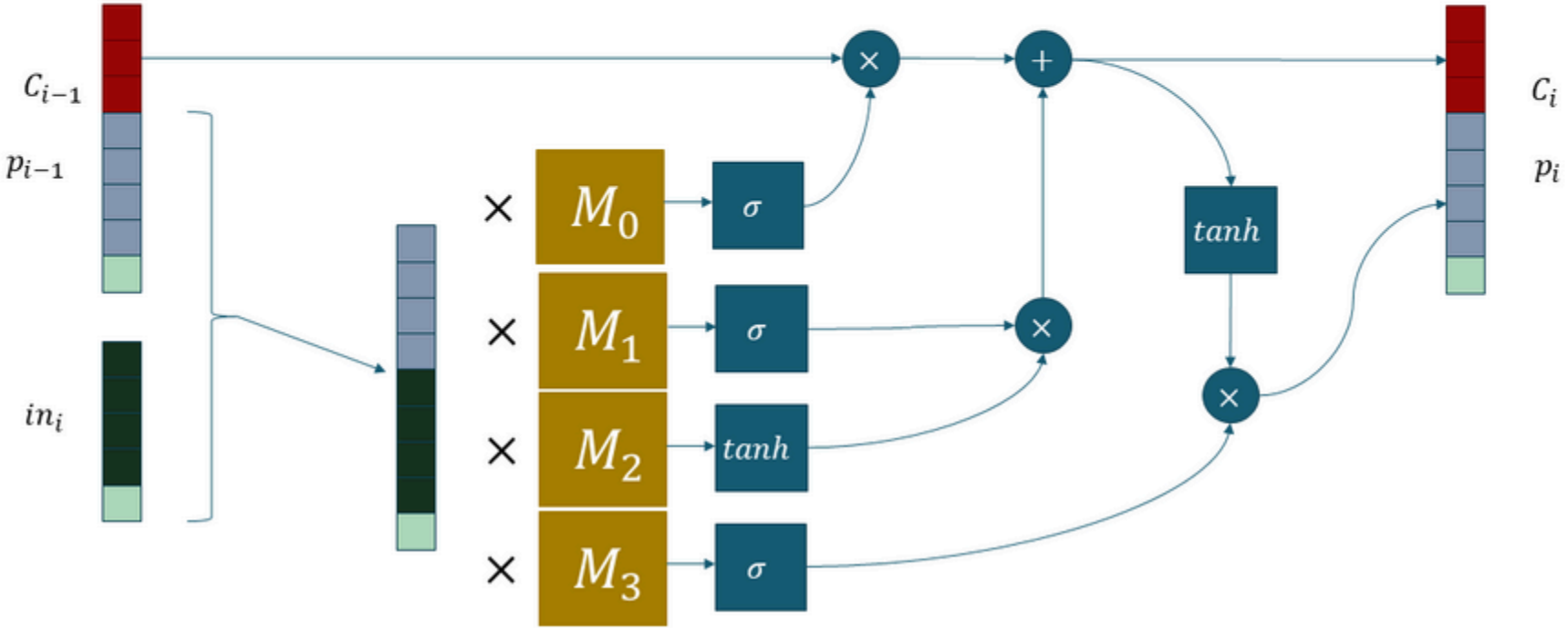
Simple recurrent neural network



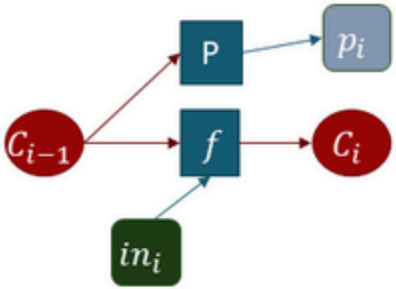


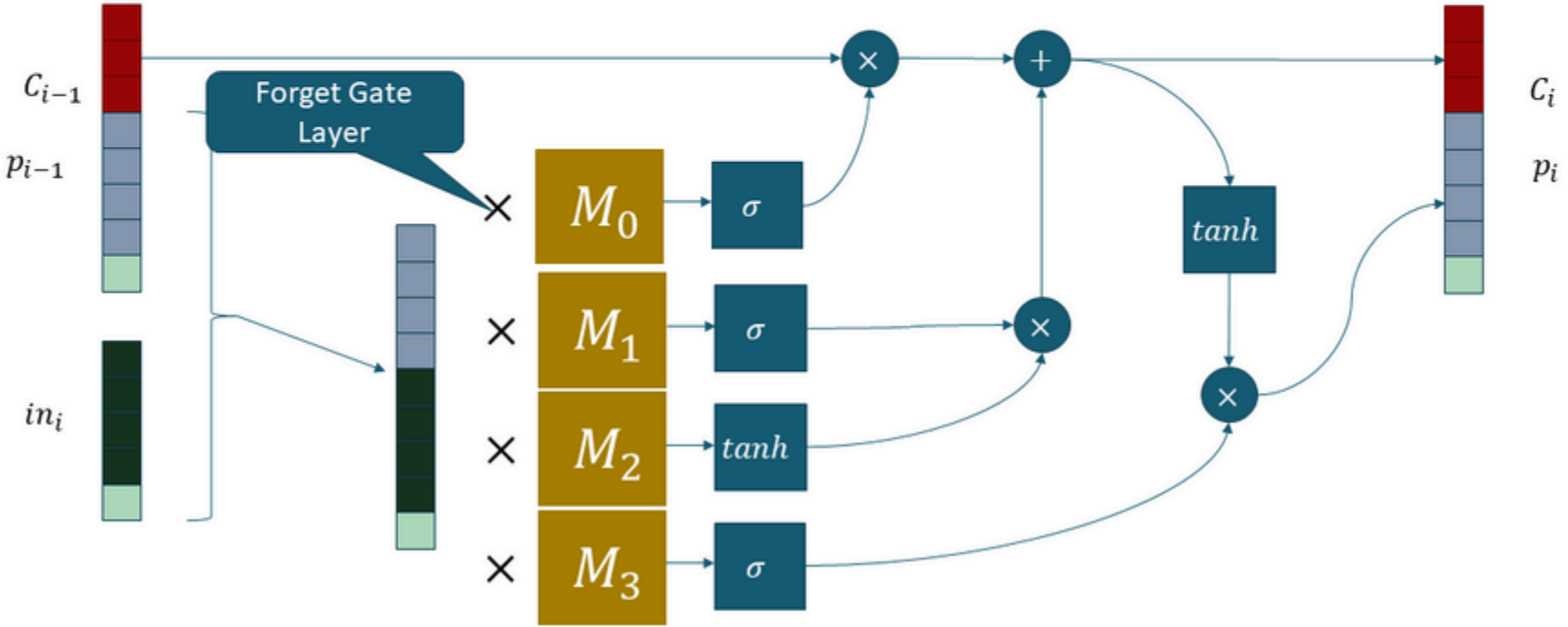
LSTM


P
p_i
C_{i-1}
f
C_i
in_i

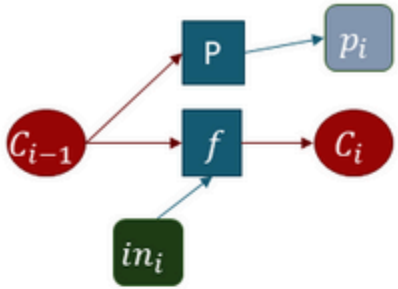

C_{i-1}
p_{i-1}
in_i
M_0
M_1
M_2
M_3
σ
σ
$tanh$
σ
$tanh$
C_i
p_i

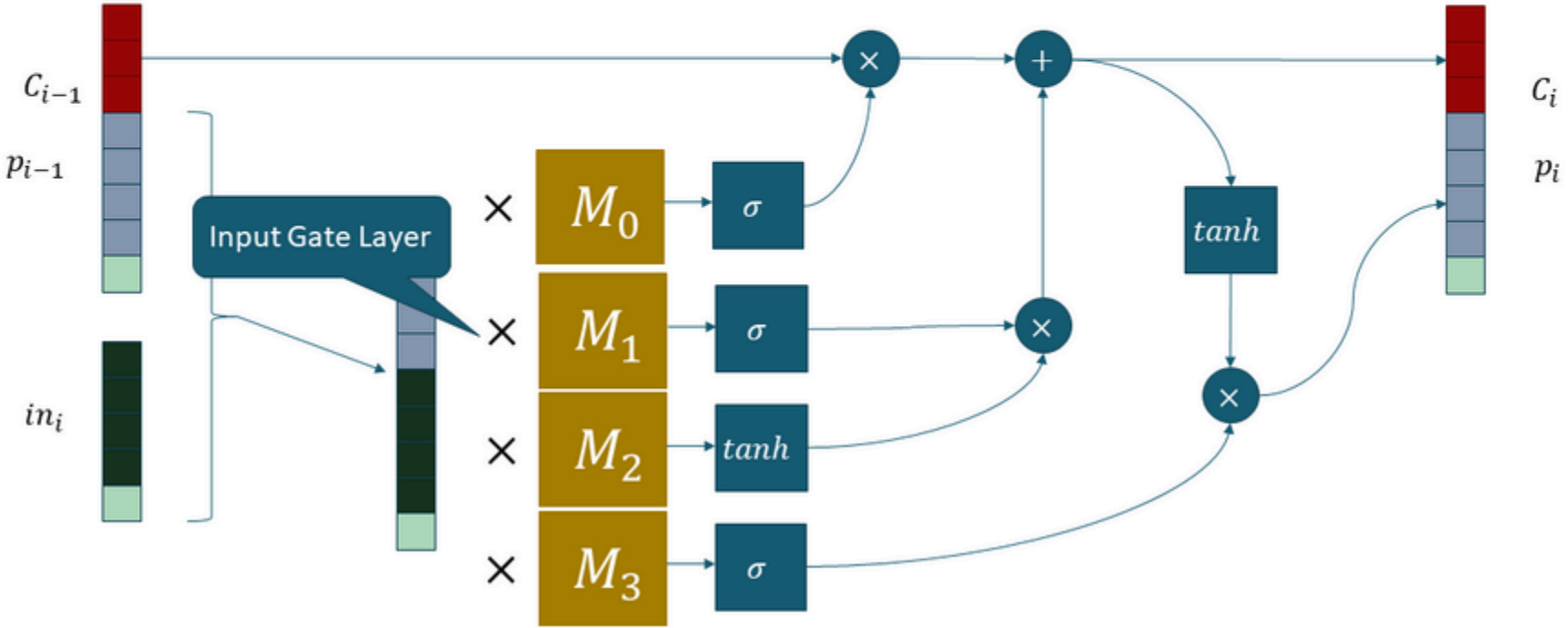
LSTM


P
p_i
C_{i-1}
f
C_i
in_i


C_{i-1}
p_{i-1}
in_i
Forget Gate Layer
M_0
M_1
M_2
M_3
σ
σ
$tanh$
σ
$tanh$
C_i
p_i

LSTM


P
p_i
C_{i-1}
f
C_i
in_i


C_{i-1}
p_{i-1}
in_i
Input Gate Layer
M_0
M_1
M_2
M_3
σ
σ
$tanh$
σ
$tanh$
C_i
p_i

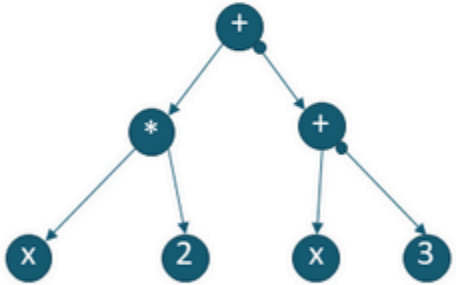
Context sensitive P G

$$
\begin{aligned}
expr = \; & expr_1 + expr_2 && A \\
| \; & expr_3 - expr_4 && B \\
| \; & expr_5 * expr_6 && C \\
| \; & var && D \\
| \; & num && E
\end{aligned}
$$

	A	B	C	D	E
Root	0.2	0.2	0.2	0.2	0.2
1	0.2	0.2	0.2	0.2	0.2
2	0	0	1/3	1/3	1/3
3	0.2	0.2	0.2	0.2	0.2
4	0	0	1/3	1/3	1/3
5	0.2	0.2	0.2	0.2	0.2
6	0.25	0.25	0	0.25	0.25

Generalizes n-gram for n = 1

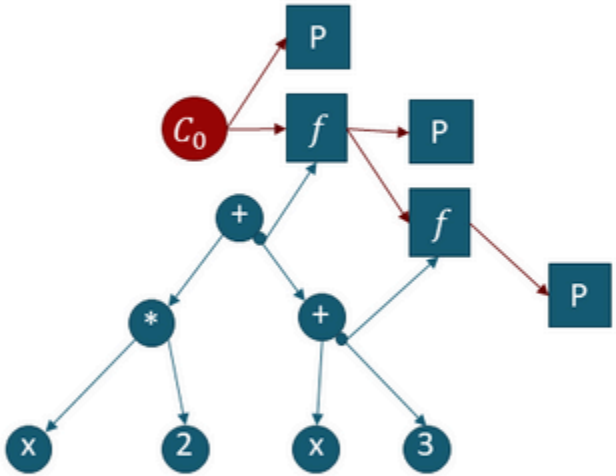
Can be extended to bigger contexts



$$P(production \mid site_i, site_{i-1})$$

Can also be generalized to a recurrent model

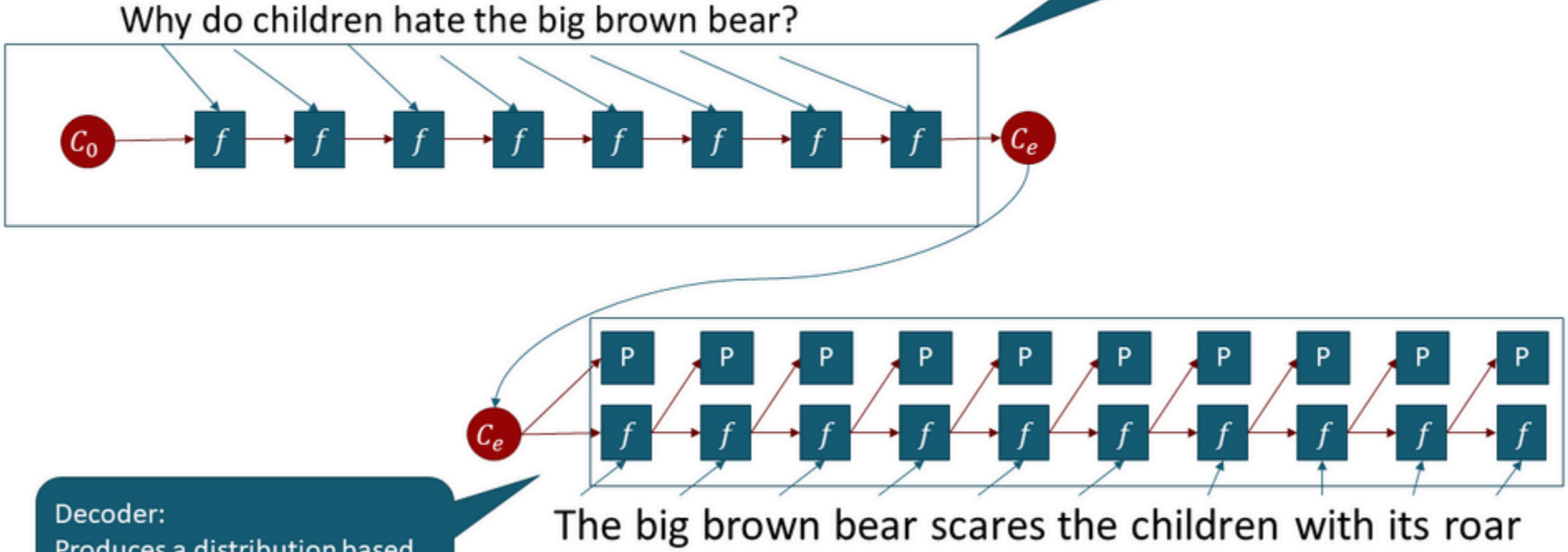
Context sensitive P G



Can also be generalized to a recurrent model

Encoder-decoder model

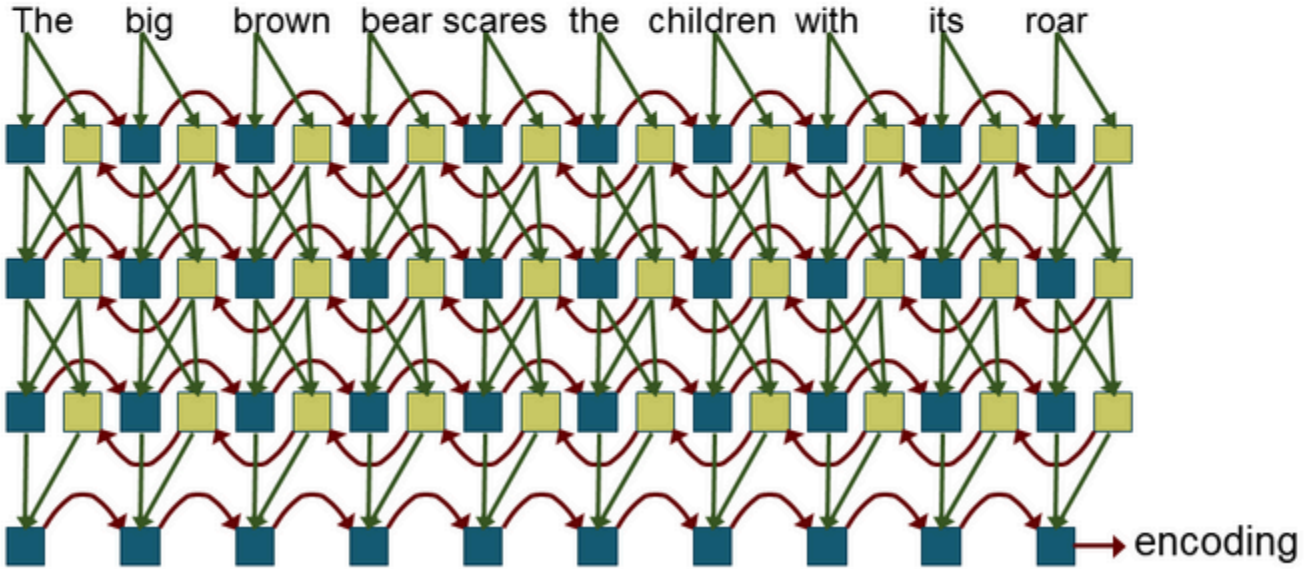
Encoder:
Generates latent representation of the evidence



Decoder:
Produces a distribution based on the evidence

Multilayer and bidirectional models

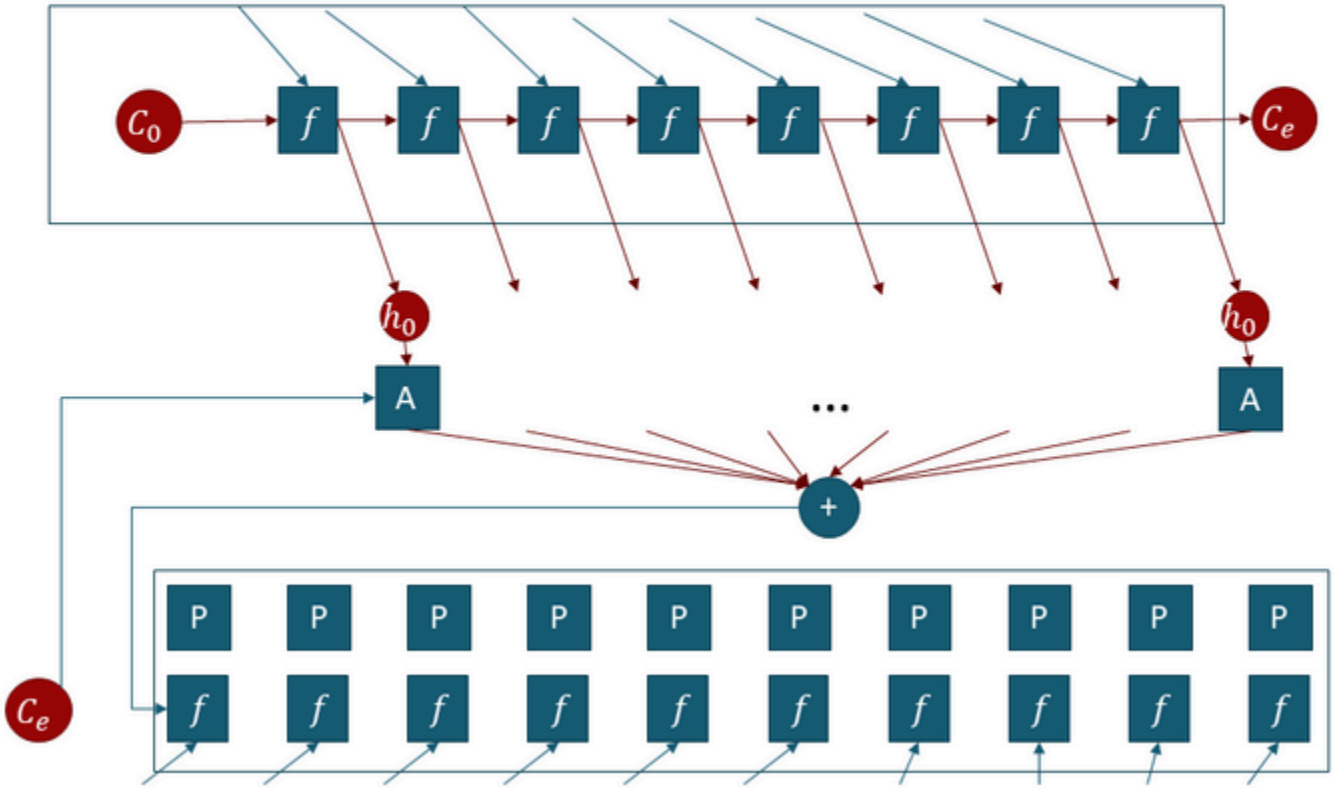
Bidirectional networks propagate information back-to-front as well as front-to-back. They can also have more than one layer.



Attention

Key idea: Summarizing into a single vector is a big bottleneck.
Every output should have direct access to the whole input

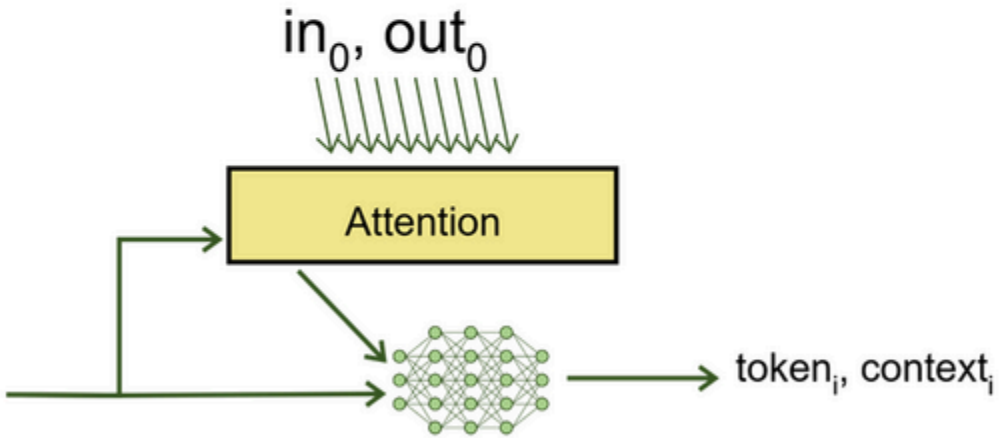
Attention



RobustFill

Single example case

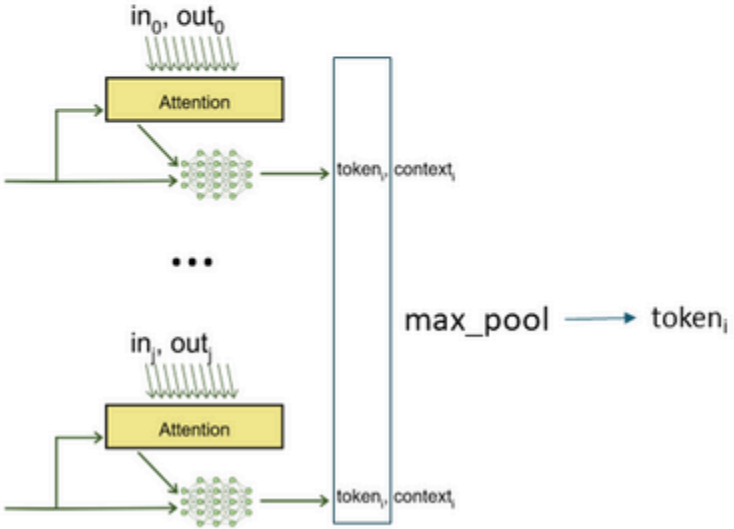
- Attend over the input/output to produce each token



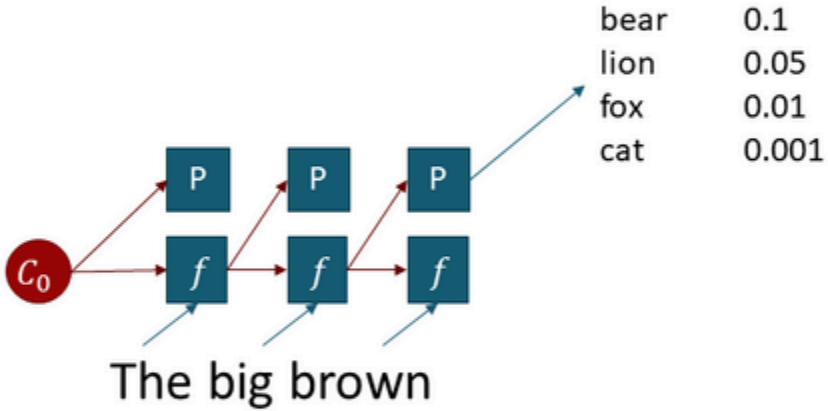
RobustFill

Multiple example case

- Compute each example independently
- pool over the result to decide which token to output



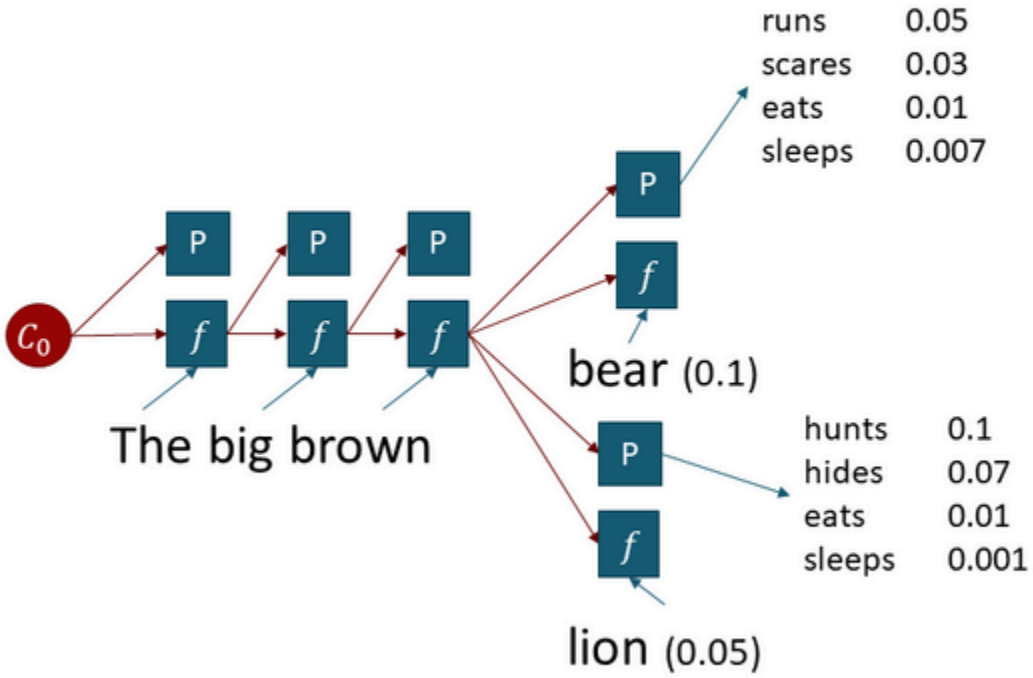
Beam Search



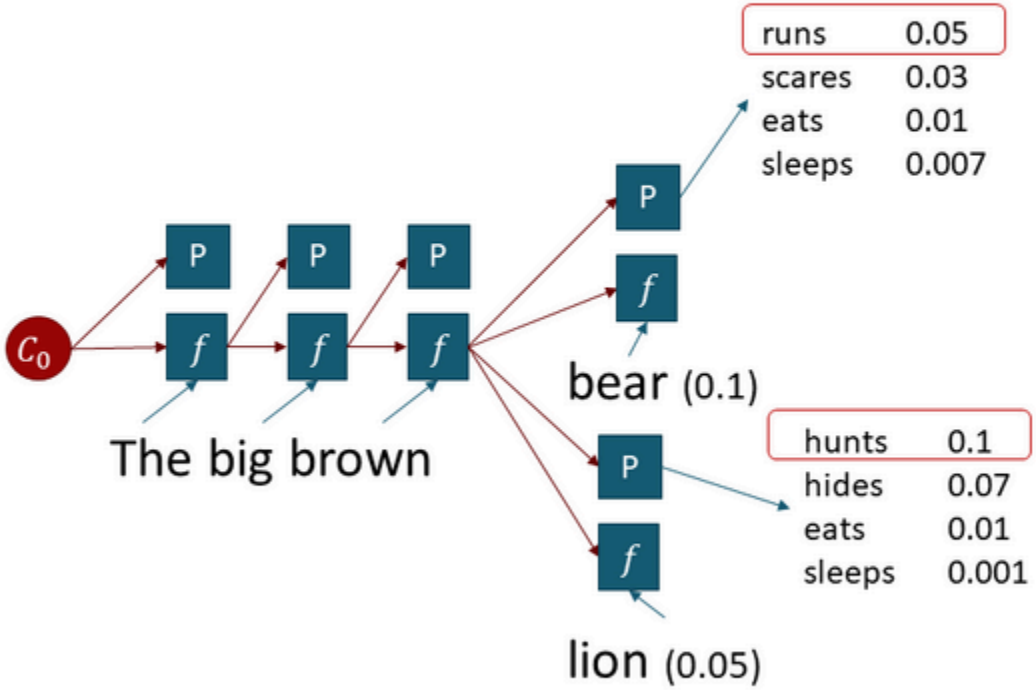
bear	0.1
lion	0.05
fox	0.01
cat	0.001

At each step pick the K most likely successors

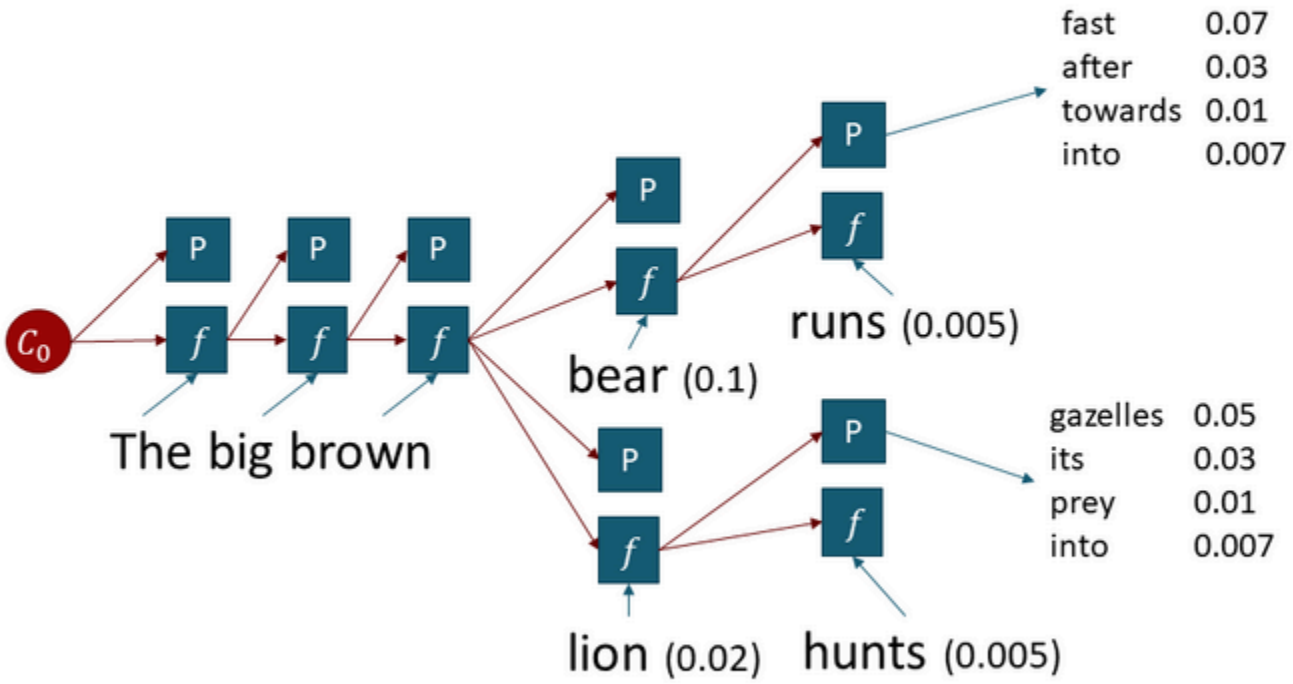
Beam Search


runs 0.05
scares 0.03
eats 0.01
sleeps 0.007
P
P
P
P
c_0
f
f
f
f
bear (0.1)
The big brown
P
hunts 0.1
hides 0.07
eats 0.01
sleeps 0.001
f
lion (0.05)

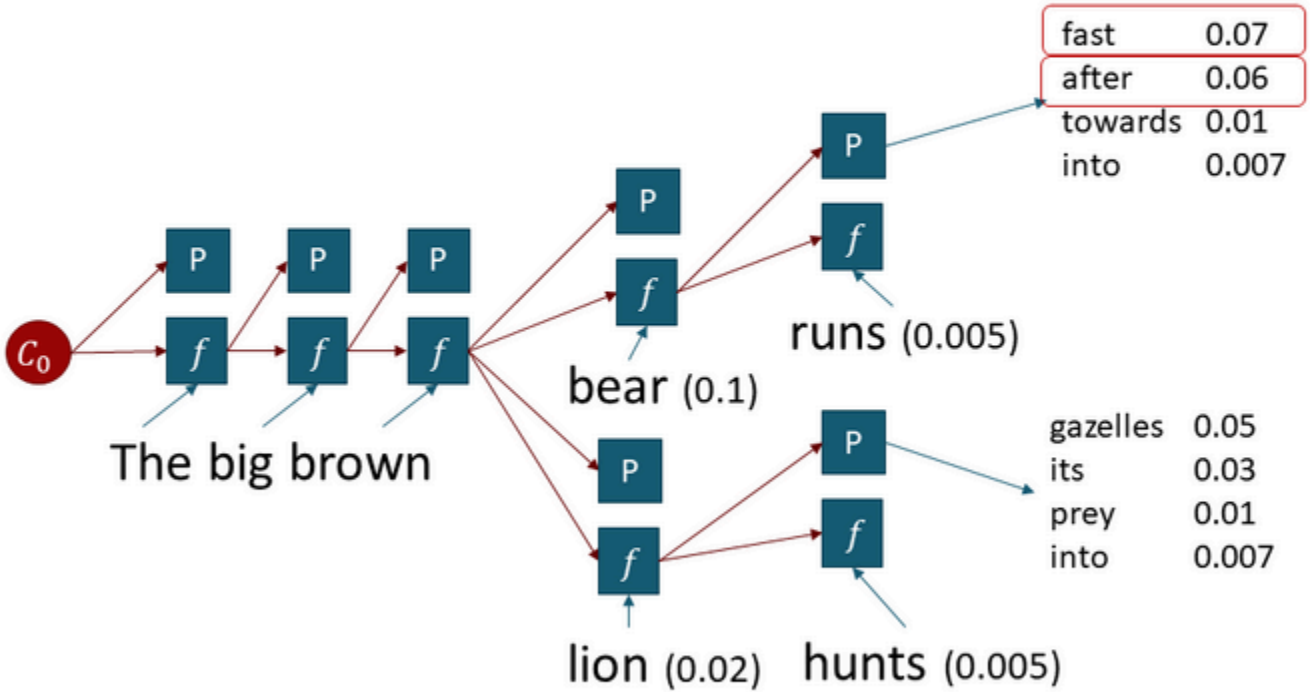
Beam Search


runs 0.05
scares 0.03
eats 0.01
sleeps 0.007
P
P
P
P
c_0
f
f
f
f
bear (0.1)
The big brown
hunts 0.1
hides 0.07
eats 0.01
sleeps 0.001
P
f
lion (0.05)

Beam Search


c_0
P
f
The big brown
bear (0.1)
runs (0.005)
lion (0.02)
hunts (0.005)
fast 0.07
after 0.03
towards 0.01
into 0.007
gazelles 0.05
its 0.03
prey 0.01
into 0.007

Beam Search


c_0
P
f
The big brown
bear (0.1)
lion (0.02)
runs (0.005)
hunts (0.005)
fast 0.07
after 0.06
towards 0.01
into 0.007
gazelles 0.05
its 0.03
prey 0.01
into 0.007

Beam search for programs

Similar to top-down search

- Same strategies for pruning the search space apply

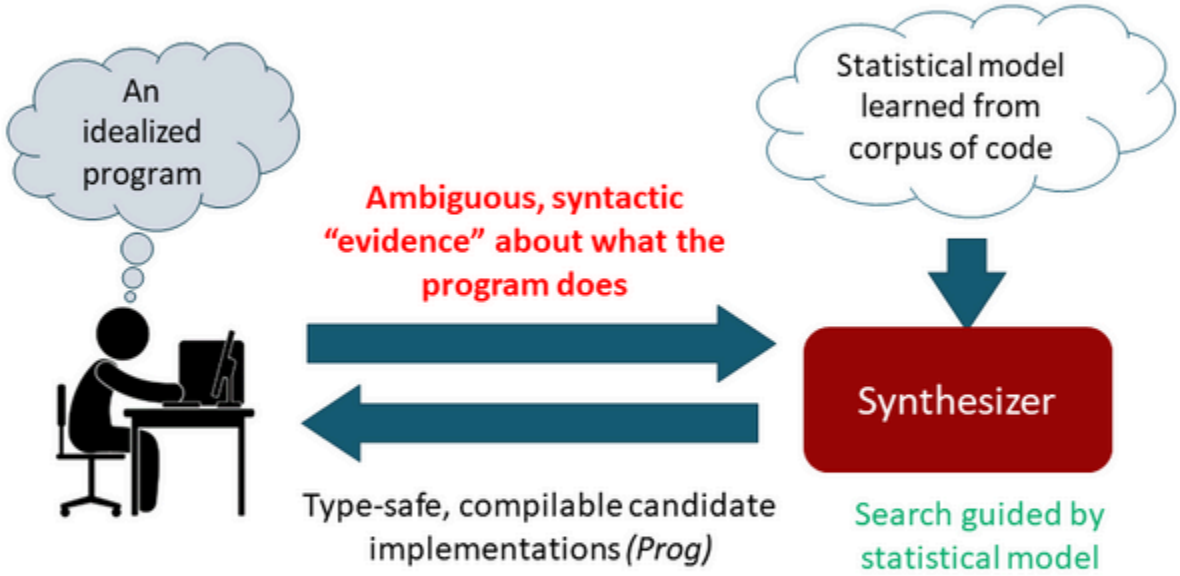
Bayesian View of PBE

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

$$P(f \mid evidence) = \frac{P(evidence \mid f)P(f)}{P(evidence)}$$

Can we learn $P(f|evidence)$ directly?

Synthesis from ambiguous evidence



Neural Sketch Learning for Conditional Program Generation. Murali, Qi, Chaudhuri, and Jermaine. ICLR 2018 (oral presentation).

The Bayou system: Example

Input

```
void read(String file) {
   /// call:readline
   /// type:BufferedReader

}
```



Output

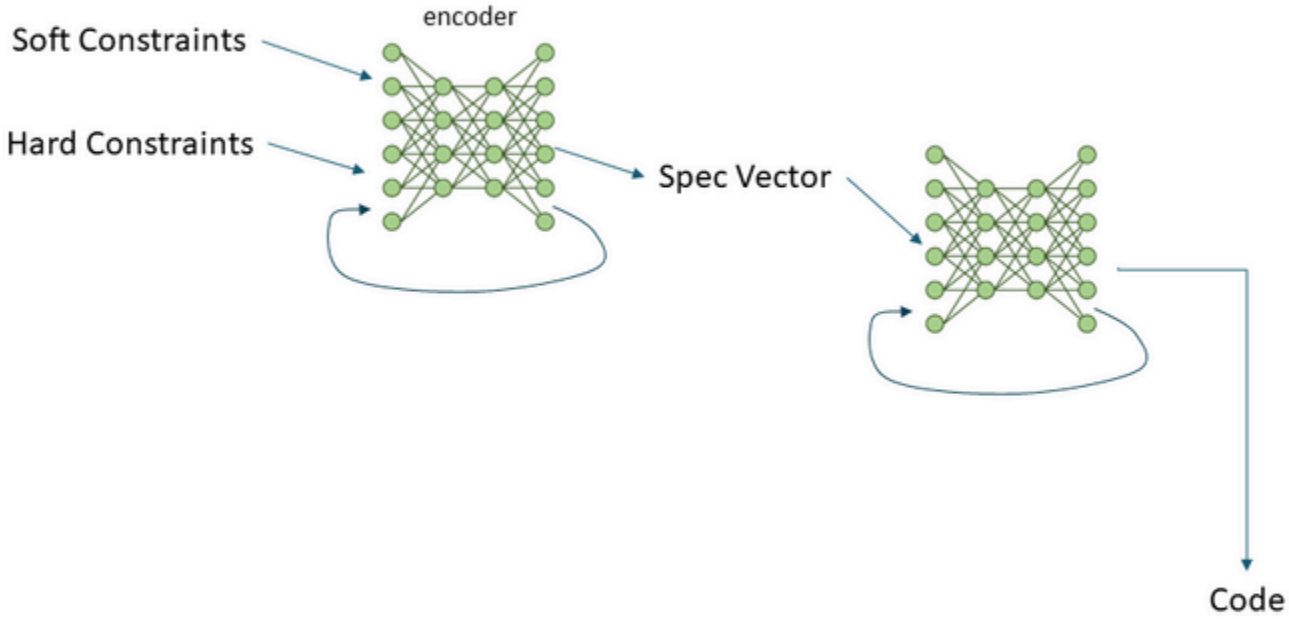
```
void read(String file) {

   String s;
   BufferedReader br;
   FileReader fr;
    try {
       fr = new FileReader(file);

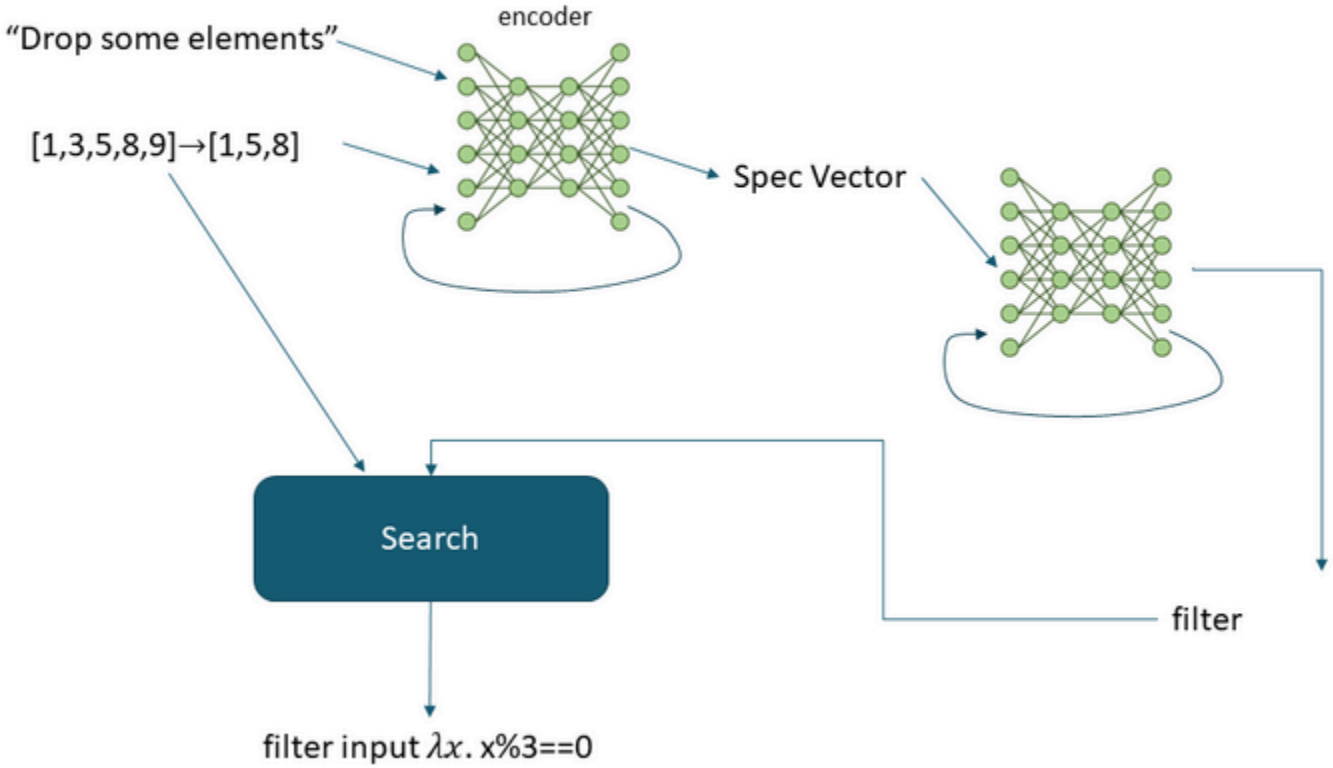
       br = new BufferedReader(fr);

       while ((s = br.readLine())!=null){
        }
       br.close();
    }
   catch (FileNotFoundException _e) {}
   catch (IOException _e) {}
}
```

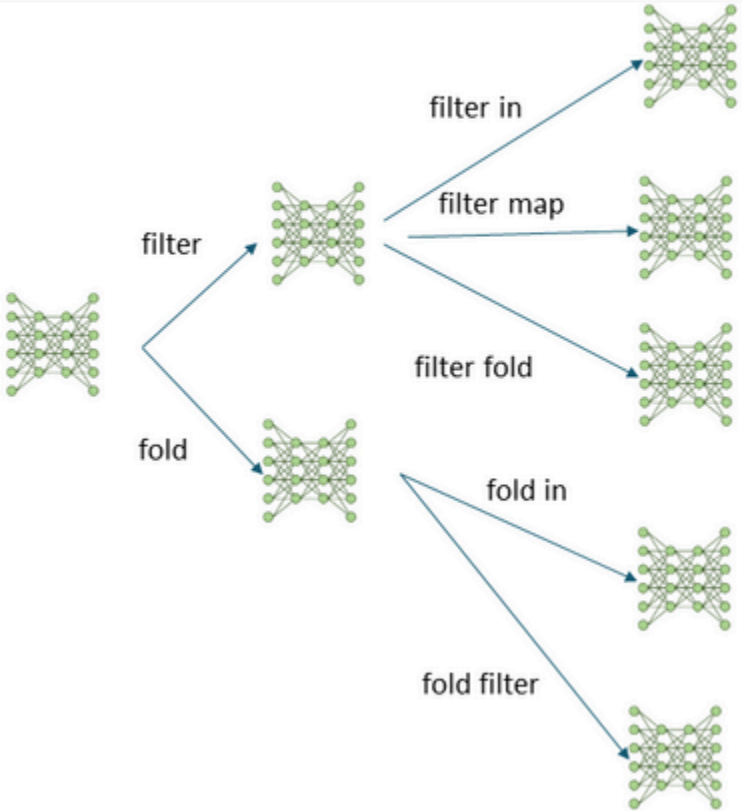
Learning to Sketch



Learning to Sketch



Beam Search + Synthesis



Training

Key idea

Penalize heavily for incorrect token
Penalty for hole depends on cost of completing it

"Drop some elements"

[1,3,5,8,9]→[1,5,8]

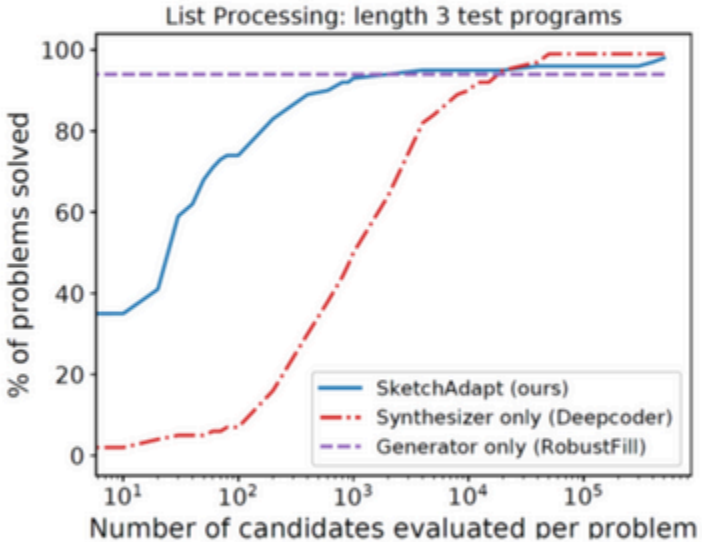


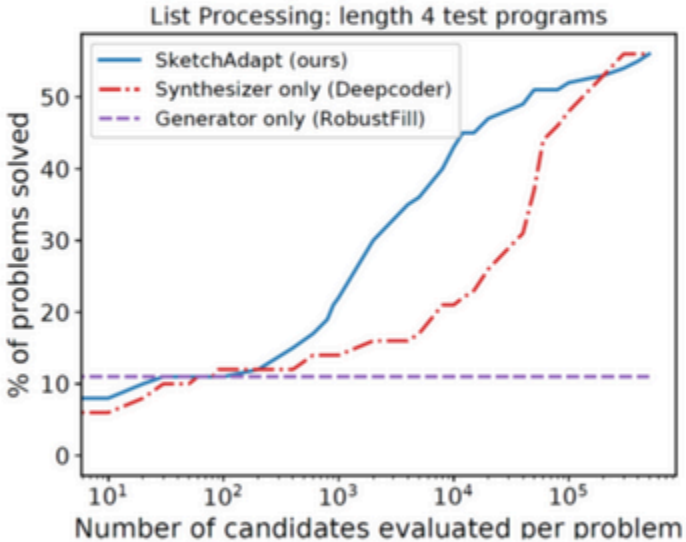
filter input λx. x%3==0

fold input λx. x%3==0 ✗

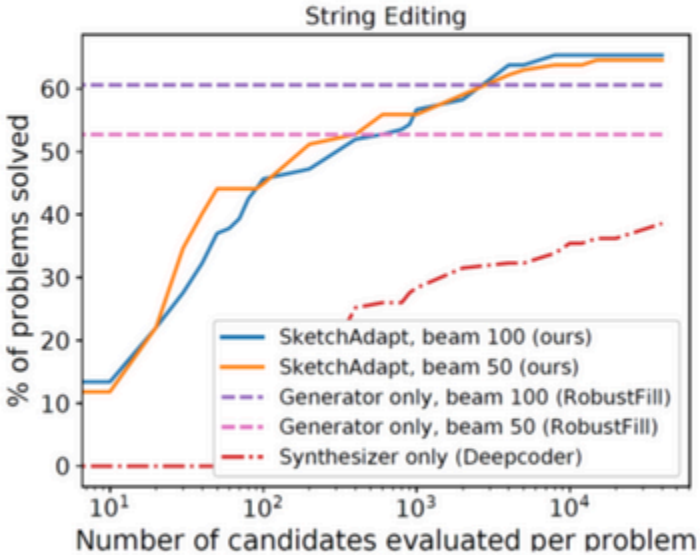
filter input HOLE λx. HOLE

Results

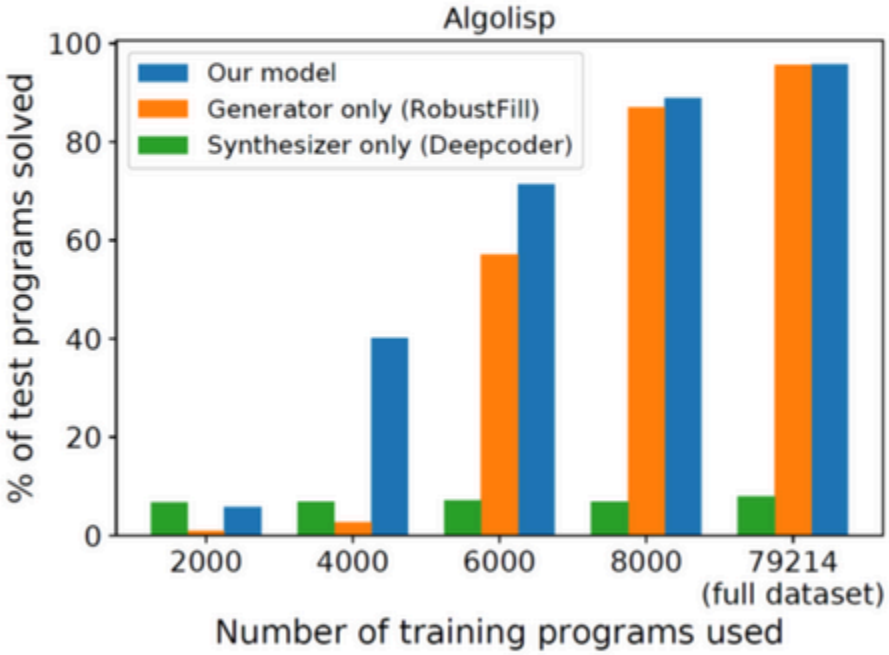

List Processing: length 3 test programs
% of problems solved
Number of candidates evaluated per problem
SketchAdapt (ours)
Synthesizer only (Deepcoder)
Generator only (RobustFill)


List Processing: length 4 test programs
% of problems solved
Number of candidates evaluated per problem
SketchAdapt (ours)
Synthesizer only (Deepcoder)
Generator only (RobustFill)

Results


String Editing
% of problems solved
0
10
20
30
40
50
60
10^1
10^2
10^3
10^4
Number of candidates evaluated per problem
SketchAdapt, beam 100 (ours)
SketchAdapt, beam 50 (ours)
Generator only, beam 100 (RobustFill)
Generator only, beam 50 (RobustFill)
Synthesizer only (Deepcoder)

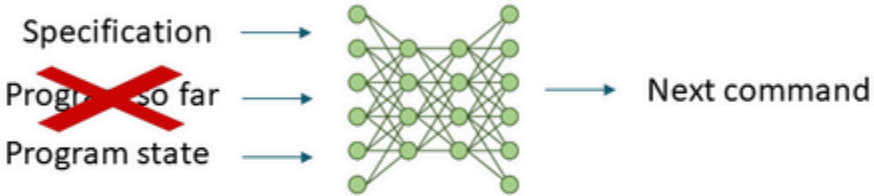
Algolisp Results

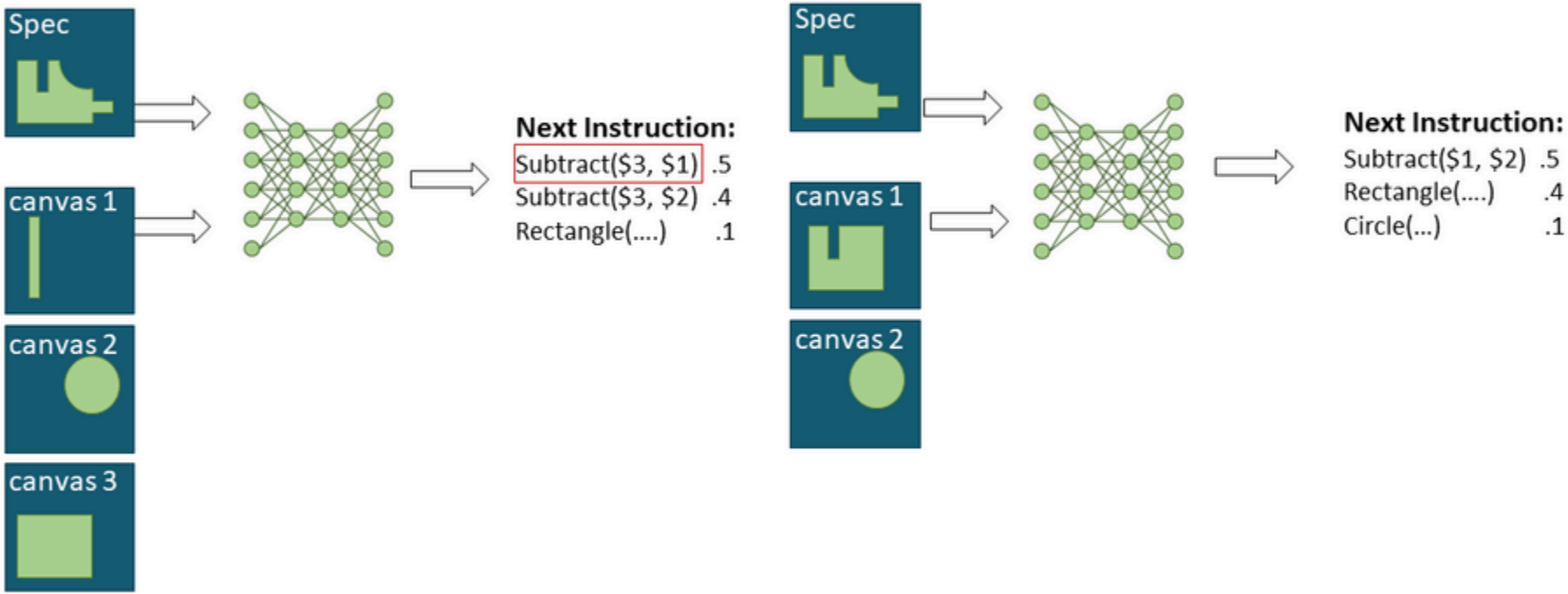


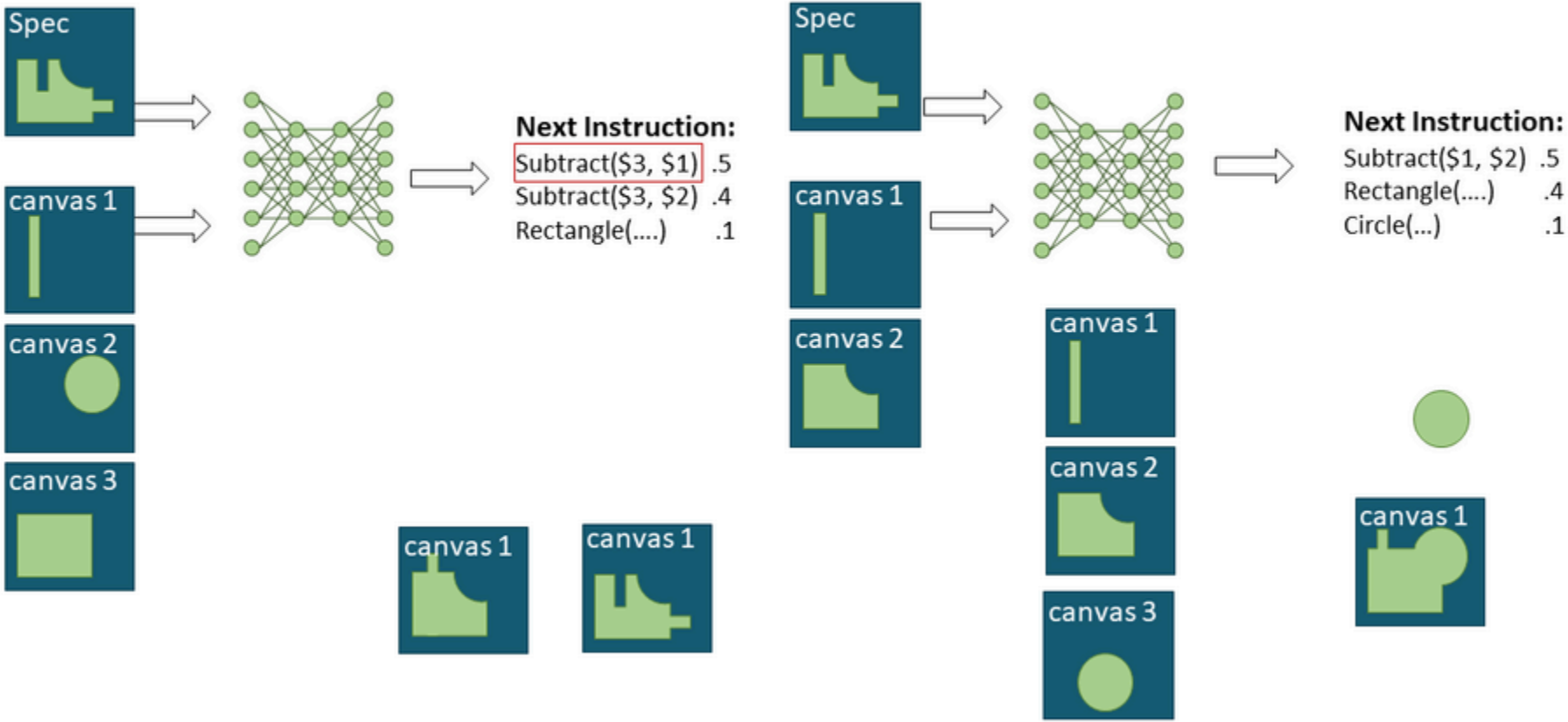
Generating programs



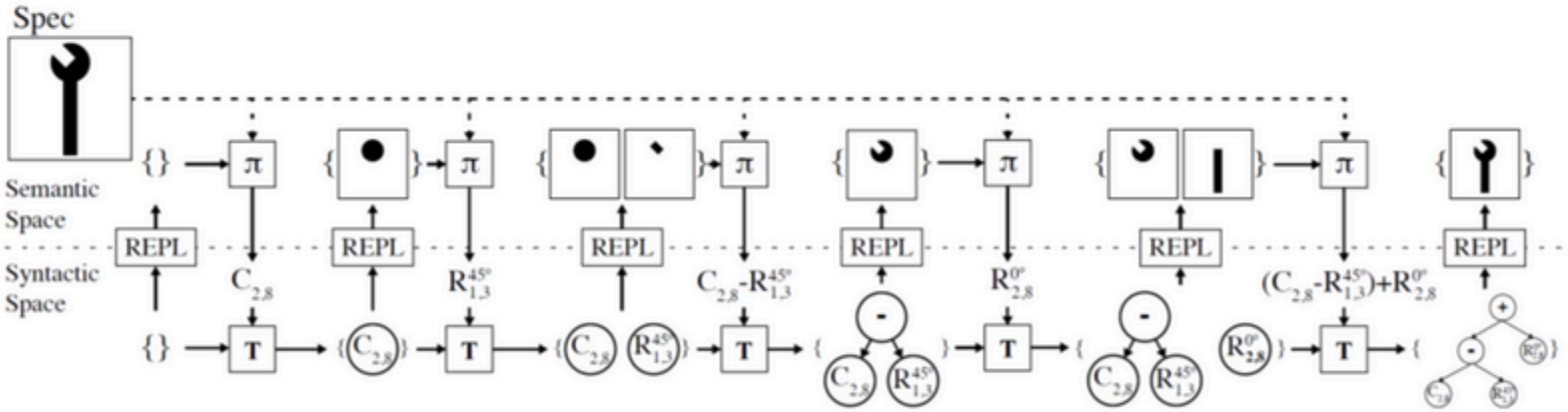
Generating programs



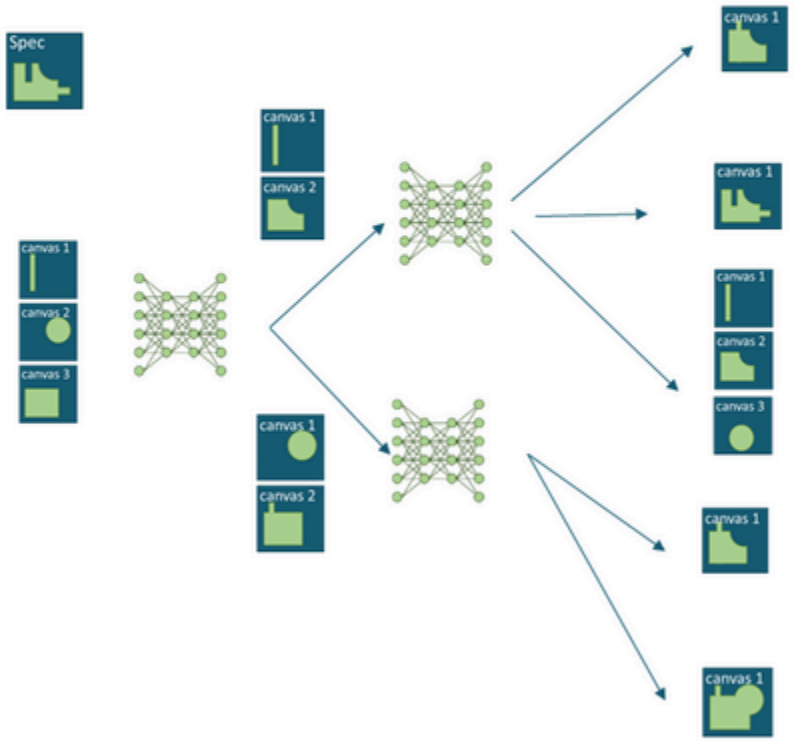

Spec
canvas 1
canvas 2
canvas 3
Next Instruction:
Subtract($3, $1) .5
Subtract($3, $2) .4
Rectangle(....) .1
Spec
canvas 1
canvas 2
Next Instruction:
Subtract($1, $2) .5
Rectangle(....) .4
Circle(...) .1


Spec
canvas 1
canvas 2
canvas 3
Next Instruction:
Subtract($3, $1) .5
Subtract($3, $2) .4
Rectangle(....) .1
canvas 1
canvas 1
Spec
canvas 1
canvas 2
canvas 1
canvas 2
canvas 3
Next Instruction:
Subtract($1, $2) .5
Rectangle(....) .4
Circle(...) .1
canvas 1

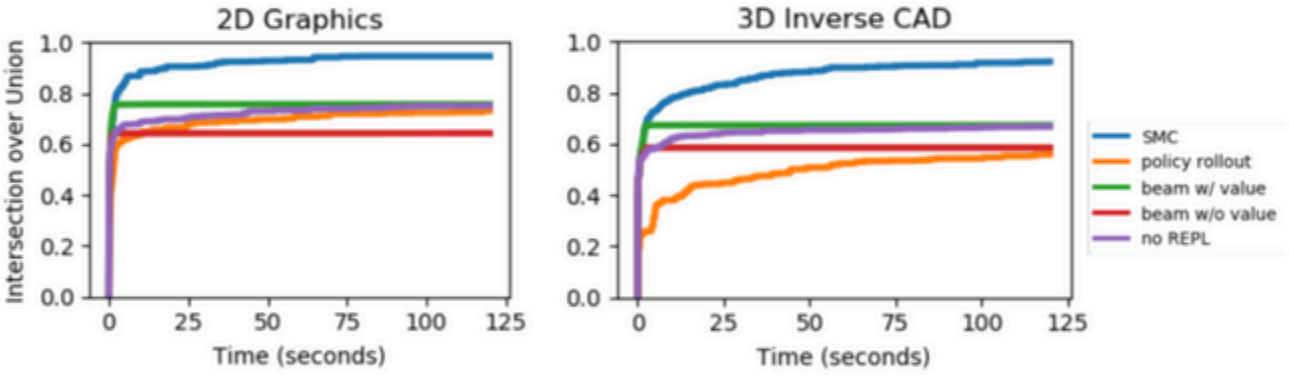
REPL

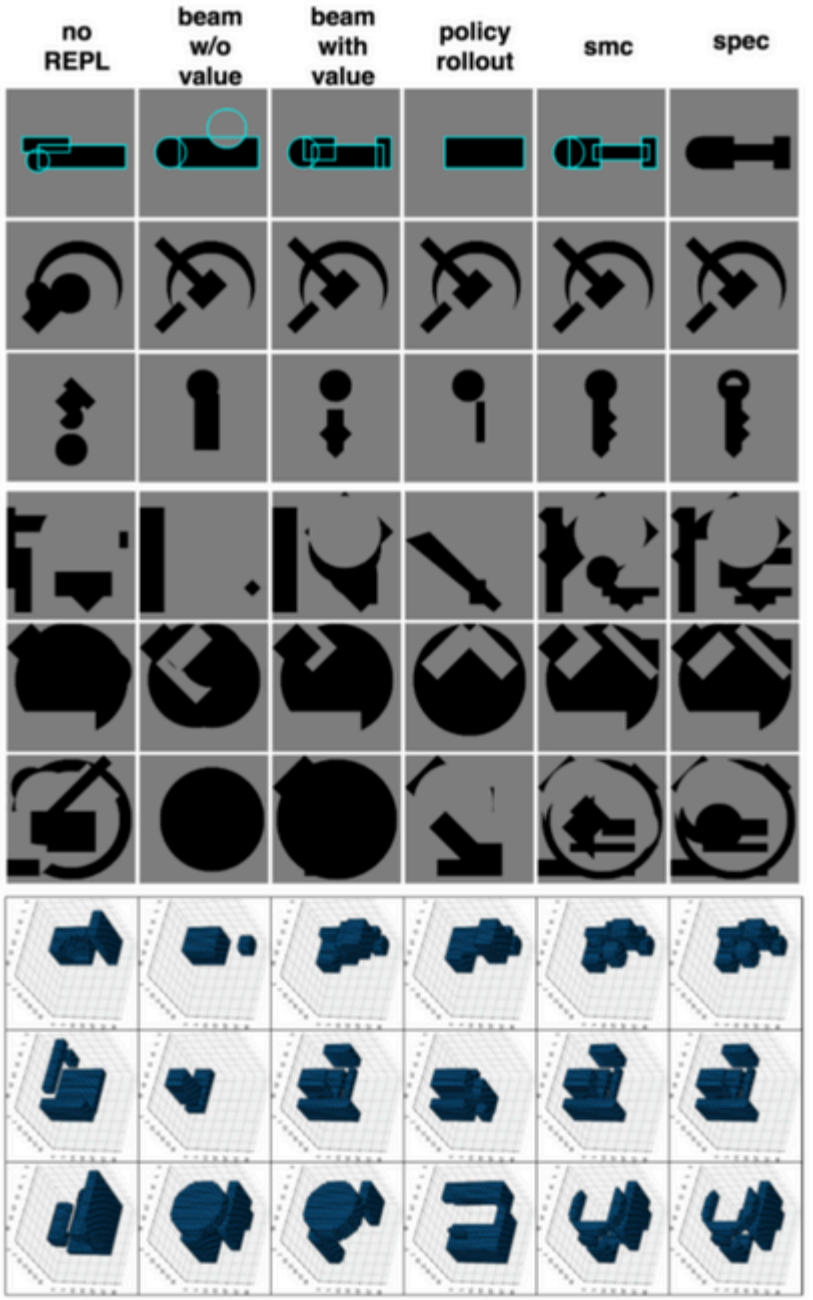

Spec
Semantic Space
Syntactic Space
REPL
π
T
$C_{2,8}$
$R^{45°}_{1,3}$
$C_{2,8}$-$R^{45°}_{1,3}$
$R^{0°}_{2,8}$
($C_{2,8}$-$R^{45°}_{1,3}$)+$R^{0°}_{2,8}$

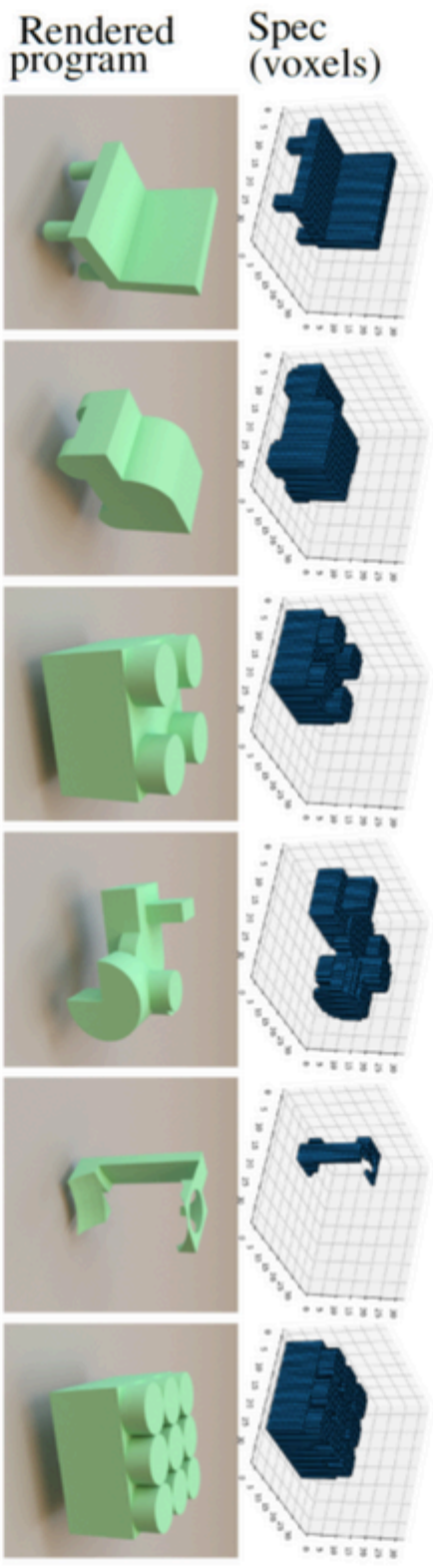
Value function over *program states*



CSG domain


2D Graphics
3D Inverse CAD
Intersection over Union
Time (seconds)
Time (seconds)
1.0
0.8
0.6
0.4
0.2
0.0
0
25
50
75
100
125
SMC
policy rollout
beam w/ value
beam w/o value
no REPL


no REPL
beam w/o value
beam with value
policy rollout
smc
spec


Rendered program
Spec (voxels)